

LCQ: Low-Rank Codebook based Quantization for Large Language Models

Anonymous ACL submission

Abstract

Large language models (LLMs) have recently demonstrated promising performance in many tasks. However, the high storage and computational cost of LLMs has become a challenge for deploying LLMs. Weight quantization has been widely used for model compression, which can reduce both storage and computational cost. Most existing weight quantization methods for LLMs use a rank-one codebook for quantization, which result in substantial accuracy loss when the compression ratio is high. In this paper, we propose a novel weight quantization method, called low-rank codebook based quantization (LCQ), for LLMs. LCQ adopts a low-rank codebook, the rank of which can be larger than one, for quantization. Experiments show that LCQ can achieve better accuracy than existing methods with a negligibly extra storage cost.

1 Introduction

Large language models (LLMs), such as GPT (Brown et al., 2020), GLM (Zeng et al., 2023), LLaMA (Touvron et al., 2023) and LLaVA (Liu et al., 2023), have recently demonstrated promising performance in many tasks including both understanding and generation of natural language, image, video and speech. The number of parameters in LLMs is typically very large. For example, the number of parameters in GPT (Brown et al., 2020) can reach up to 175 billion. A large number of parameters will result in high storage and computational cost, which has become a challenge for deploying LLMs in real applications.

Weight quantization (Lin et al., 2023; Frantar et al., 2023; Dettmers et al., 2023; Xu et al., 2024), which represents (quantizes) each high bit-width weight by a low bit-width value, has been widely used for model compression. Weight quantization can reduce both storage and computational

cost for model inference (Frantar et al., 2023), and hence can facilitate model deployment in real applications. Naive weight quantization will typically cause accuracy degradation, especially for cases when the compression ratio is high, i.e., the bit-width after quantization is too low. Existing weight quantization methods for LLMs have proposed different techniques to mitigate the accuracy degradation caused by quantization. For example, GPTQ (Frantar et al., 2023) uses output reconstruction error rather than simple parameter reconstruction error for quantization. GPTQ (Frantar et al., 2023) primarily focus on learning quantization indices and do not perform additional optimizations for the quantization codebook. AWQ (Lin et al., 2023) and OmniQuant (Shao et al., 2024) mainly focus on learning quantization codebook, achieving better results than GPTQ (Frantar et al., 2023). Although existing weight quantization methods can achieve accuracy comparable to the original model with 4-bit or higher-bit quantization, there remains a significant accuracy degradation for quantization when the bit-width is lower than 4.

One common characteristic of most existing weight quantization methods for LLMs is that they use a rank-one codebook for quantization. Specifically, the quantization codebook for weights can be seen as the outer product of a scaling vector (Lin et al., 2023; Frantar et al., 2023; Dettmers et al., 2023) and a quantization point set (QPS) vector (Oh et al., 2022). Each element in the scaling vector corresponds to a scaling factor for a group of weights and the QPS vector is constructed from quantiles of uniform distribution (Frantar et al., 2023; Lin et al., 2023) or Gaussian distribution (Dettmers et al., 2023). Although rank-one codebook can reduce the storage cost of the model, it has limited representation ability. This might be one of the main reasons to explain the significant accuracy degradation for existing quantization methods when the bit-width is lower than 4.

In this paper, we propose a novel weight quantization method, called low-rank codebook based quantization (LCQ), for LLMs. The main contributions of LCQ are outlined as follows:

- LCQ adopts a low-rank codebook, the rank of which can be larger than one, for quantization.
- In LCQ, a gradient-based optimization algorithm is proposed to optimize the parameters of the codebook.
- LCQ adopts a double quantization strategy for compressing the parameters of the codebook, which can reduce the storage cost of the codebook.
- Experiments show that LCQ can achieve better accuracy than existing methods with a negligible extra storage cost.

2 Related Work

Existing weight quantization methods for LLMs typically employ post-training quantization (PTQ) which is relatively fast and does not require access to the full original training data (Frantar et al., 2023; Lin et al., 2023; Dettmers et al., 2023). PTQ methods such as BRECQ (Li et al., 2021), OBC (Frantar and Alistarh, 2022) and QDrop (Wei et al., 2022) are designed for quantizing traditional non-LLMs, which cannot be directly used for LLMs due to their high optimization cost. Recently, some PTQ methods are specially designed for LLM quantization. GPTQ (Frantar et al., 2023) primarily focuses on learning quantization indices and uses output reconstruction error as objective function. But GPTQ does not perform optimization for the quantization codebook, which results in significant accuracy loss with low bit-width. AWQ (Lin et al., 2023) and OmniQuant (Shao et al., 2024) mainly focus on learning the quantization codebook, and the quantization indices are directly computed based on the index of the quantization value that is nearest to the full-precision weight. The codebooks of AWQ and OmniQuant are rank-one, limiting the representation ability. Very recently, there appear some PTQ methods like OWQ (Lee et al., 2024) and SpQR (Dettmers et al., 2024), which mainly address the issue of outliers in activations by assigning higher bit-widths to the channels of activation outliers. Our LCQ is orthogonal to the methods like OWQ and SpQR, and can be further integrated with these methods.

Moreover, some LLMs compression methods (Dettmers et al., 2022; Xiao et al., 2023; Yuan et al., 2023; Wei et al., 2023) integrate weight quantization and activation quantization for further accelerating LLMs inference. However, the accuracy drops significantly when quantizing both weights and activations.

PTQ methods have also been integrated with parameter-efficient fine tuning (PEFT). Representative methods include QLoRA (Dettmers et al., 2023), PEQA (Kim et al., 2023), QA-LoRA (Xu et al., 2024), and LoftQ (Li et al., 2024). But the main goal of these methods is for PEFT rather than directly quantizing the original LLMs. Our LCQ can also be used for PEFT, but this is not the focus of this paper.

3 Methodology

As in existing weight quantization methods for LLMs (Frantar et al., 2023; Lin et al., 2023; Shao et al., 2024), we also employ post-training quantization in our LCQ. In post-training quantization, we have a calibration dataset, which is denoted as $\mathcal{X} = \{\mathbf{X}_1, \dots, \mathbf{X}_i, \dots, \mathbf{X}_{N_X}\}$. Here, each calibration sample $\mathbf{X}_i \in \mathbb{R}^{L \times D}$ has a sequence length of L and feature dimension of D , and N_X is the total number of calibration samples. As in AWQ (Lin et al., 2023) and OmniQuant (Shao et al., 2024), we perform grouped quantization for the weights of linear layers. More specifically, for a linear layer with N_W weights, we first divide all the weights into N_V subsets, with each subset having N_W/N_V weights. Then for each subset of weights, we group its weights into N_G groups. We denote each subset of weights as $\mathbf{W} \in \mathbb{R}^{N_G \times G}$, in which G denotes the weight group size and $N_G = \frac{N_W}{N_V \times G}$ denotes the number of weight groups per subset. A quantization codebook containing quantization values is maintained for each group of weights. For weight quantization with b bits, the number of all quantization values of each group of weights is $N_Q = 2^b$. We denote the quantization codebook of all groups as $\mathbf{C} \in \mathbb{R}^{N_G \times N_Q}$, the i th row of which is the codebook corresponding to the i th group of weights. The quantization function operates on the weights in an element-wise way, which quantizes each weight to the nearest quantization value in the codebook. Let $\mathbf{Z} \in [0, 1, \dots, N_Q]^{N_G \times G}$ denote the quantization index of $\mathbf{W}$, in which $Z_{i,j}$ denotes

the quantization index of $W_{i,j}$. We have

$$Z_{i,j} = \arg \min_{1 \leq k \leq N_Q} \|W_{i,j} - C_{i,k}\|. \quad (1)$$

The quantization function $Q(\cdot)$ can be defined as follows:

$$Q(W_{i,j}, C_i) = C_{i,Z_{i,j}}. \quad (2)$$

Here, C_i denotes the i th row of $\mathbf{C}$. $\mathbf{W}^Q = Q(\mathbf{W}, \mathbf{C})$ is the quantized weight matrix with $W_{i,j}^Q = Q(W_{i,j}, C_i)$.

Then we can use the quantized weight matrix $\mathbf{W}^Q$ for inference. In real deployment, we only need to store the codebook matrix $\mathbf{C}$ and the quantization index matrix $\mathbf{Z}$ (Han et al., 2016). From (1), we can find that $\mathbf{Z}$ is computed based on $\mathbf{C}$. Hence, $\mathbf{C}$ is the key for quantization, which will affect accuracy, storage cost and computational cost.

3.1 Low-Rank Codebook

In AWQ (Lin et al., 2023) and GPTQ (Frantar et al., 2023), the intervals between quantization values are equal, and hence it is not necessary to explicitly store the quantization codebook $\mathbf{C}$. More specifically, $\mathbf{C}$ can be represented by two vectors, a scaling vector $\mathbf{S}_1 \in \mathbb{R}^{1 \times N_G}$ and a fixed quantization point set (QPS) vector $\mathbf{V}_1 \in \mathbb{R}^{1 \times N_Q}$. $\mathbf{V}_1 = [-1, -1 + \frac{2}{2^b-1}, \dots, 1 - \frac{2}{2^b-1}, 1]$ is a uniformly spaced vector between $[-1, 1]$. $\mathbf{S}_1$ can be adaptively obtained based on the weight matrix $\mathbf{W}$. The codebook is then obtained by computing the outer product of $\mathbf{S}_1$ and $\mathbf{V}_1$. Furthermore, to adapt to asymmetric weight distribution, a quantization offset $\mathbf{B} \in \mathbb{R}^{N_G \times 1}$ is introduced, with each group of weights having its own quantization offset. The final quantization codebook can be formulated as $\mathbf{C} = \mathbf{S}_1^T \mathbf{V}_1 - \mathbf{B}$, which is a rank-one matrix. The storage cost for $\mathbf{S}_1$, $\mathbf{V}_1$ and $\mathbf{B}$ is $O(2 * N_G + N_Q)$, which is much lower than $O(N_G * N_Q)$ for storing $\mathbf{C}$. Although rank-one codebook can reduce the storage cost, it has limited representation ability.

To improve the representation ability of the codebook, we propose to use low-rank codebook for quantization. More specifically, we introduce two matrices: $\mathbf{S} = [\mathbf{S}_1; \dots; \mathbf{S}_{N_D}] \in \mathbb{R}^{N_D \times N_G}$, $\mathbf{V} = [\mathbf{V}_1; \dots; \mathbf{V}_{N_D}] \in \mathbb{R}^{N_D \times N_Q}$. The quantization codebook $\mathbf{C}$ is obtained as follows:

$$\mathbf{C} = \Phi(\mathbf{S}, \mathbf{V}, \mathbf{B}) = \mathbf{S}^T \mathbf{V} - \mathbf{B}. \quad (3)$$

When $N_D > 1$, the rank of the codebook $\mathbf{C}$ can be larger than one. When $N_D = 1$, the codebook

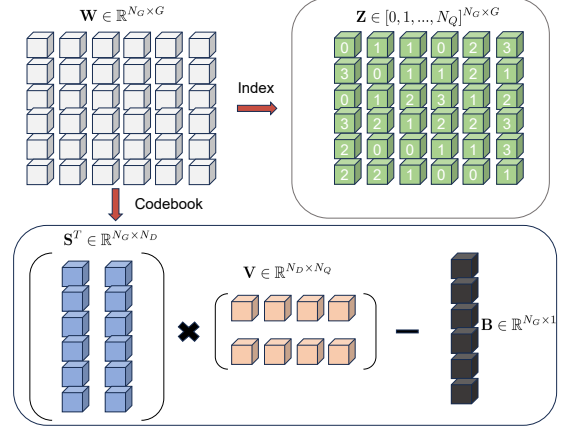


Figure 1: Illustration of low-rank codebook based quantization.

$\mathbf{C}$ degenerates to rank-one codebook as in existing methods (Lin et al., 2023; Frantar et al., 2023). Hence, our LCQ has stronger representation ability than existing methods, including existing rank-one codebook based methods as special cases. In our experiment of Section 4, we will show that LCQ with $N_D > 1$ will achieve better accuracy than LCQ with $N_D = 1$. This verifies the stronger representation ability of our low-rank codebook compared to the rank-one codebook. Figure 1 is an illustration of low-rank codebook based quantization with $N_D = 2$.

3.2 Objective Function

As illustrated in Figure 1, LCQ starts with a pre-trained model and tries to find the optimal $\mathbf{S}$, $\mathbf{V}$ and $\mathbf{B}$ for minimizing the loss caused by quantization. We use the output reconstruction error of a Transformer block as the objective function like the methods in (Bai et al., 2022; Li et al., 2021; Yao et al., 2022), which can make the parameters of a Transformer block be optimized simultaneously. Compared to the methods that use the output reconstruction error of a single linear layer as the objective function (Frantar et al., 2023; Lin et al., 2023), using the output reconstruction error of a Transformer block can avoid falling into local optima and prevent the accumulation of quantization error. A Transformer block in the Transformer-based models (Vaswani et al., 2017) typically contains the following linear layers: the query/key/value linear projection layers $\mathbf{W}^{qproj}$, $\mathbf{W}^{kproj}$, $\mathbf{W}^{vproj}$, the output projection layer $\mathbf{W}^{opproj}$ of the multi-head self-attention, and two linear layers $\mathbf{W}^{fc1}$ and $\mathbf{W}^{fc2}$ of the feed forward network. Therefore, the parameters of a

Transformer block can be represented as $\mathcal{W} = [\mathbf{W}^{qproj}, \mathbf{W}^{kproj}, \mathbf{W}^{vproj}, \mathbf{W}^{oproj}, \mathbf{W}^{fc1}, \mathbf{W}^{fc2}]$. Similarly, parameters $\mathcal{S}, \mathcal{V}, \mathcal{B}$ are used to represent all $\mathbf{S}$ s, $\mathbf{V}$ s, $\mathbf{B}$ s within a Transformer block. We define the objective function to minimize the norm of the difference between the output with quantized weights and the output with unquantized weights in a Transformer block as follows:

$$\min_{\mathcal{S}, \mathcal{V}, \mathcal{B}} \sum_{i=1}^{N_X} \|h(\mathbf{X}_i^{fp}, Q(\mathcal{W}, \Phi(\mathcal{S}, \mathcal{V}, \mathcal{B}))) - h(\mathbf{X}_i^{fp}, \mathcal{W})\|_F^2 \quad (4)$$

where $h(\cdot)$ denotes the function of a Transformer block, and the full-precision input features for the current block are denoted as $\{\mathbf{X}_1^{fp}, \dots, \mathbf{X}_{N_X}^{fp}\}$. Note that the quantization algorithm does not change the model parameters $\mathbf{W}$ (Shao et al., 2024), in order to avoid the forgetting of existing knowledge. By fixing $\mathbf{W}$, we can also reduce the optimization overhead by reducing the number of parameters to be optimized.

3.3 Gradient-based Optimization

To solve the problem in (4), we propose a gradient-based optimization algorithm with strategies of gradient approximation for the quantization function, reparameterization of quantization parameters, and initialization of quantization parameters.

3.3.1 Gradient Approximation for Quantization Function

The $\argmin(\cdot)$ operation in (1) will make the gradient unable to backpropagate through this function. This makes it challenging to learn the quantization codebook. We propose a gradient approximation strategy to address this issue.

As in N2UQ (Liu et al., 2022), we rewrite the quantization function into a form that adds up multiple segments, each corresponding to the interval between two adjacent quantization values. The form is as follows:

$$Q(W_{i,j}, \mathbf{C}_i) = C_{i,\pi_i(1)} + \sum_{k=1}^{N_Q-1} (C_{i,\pi_i(k+1)} - C_{i,\pi_i(k)}) * \chi_{0.5}^{(k)} \left(\text{clip} \left(\frac{W_{i,j} - C_{i,\pi_i(k)}}{C_{i,\pi_i(k+1)} - C_{i,\pi_i(k)}}, 0, 1 \right) \right), \quad (5)$$

where $\pi_i(\cdot)$ is a sorting function that is responsible for sorting each row of the quantization codebook $\mathbf{C}$ in an ascending order. The function $\text{clip}(\cdot, 0, 1)$

clips values outside $[0, 1]$ to be within the $[0, 1]$ range. The step function $\chi_{0.5}^{(k)}(x)$ is an element-wise function that discretizes weights, which is defined as follows:

$$\chi_{0.5}^{(k)}(x) = \begin{cases} 1 & \text{if } x > 0.5, \text{ or } (x = 0.5 \text{ and } k\%2 = 1), \\ 0 & \text{if } x < 0.5, \text{ or } (x = 0.5 \text{ and } k\%2 = 0). \end{cases} \quad (6)$$

Here, we quantize critical values that are in the middle of two quantization values to the quantization value at the even position, corresponding to the condition $k\%2 = 0$ in (6). This avoids quantizing all critical values to the smaller/larger quantization value, preventing an overall bias in the quantization results. Furthermore, the elements which make $C_{i,\pi_i(k+1)} - C_{i,\pi_i(k)}$ equal zero, i.e., two adjacent quantization values are equal, can cause division by zero. To prevent such cases, we truncate values of $C_{i,\pi_i(k+1)} - C_{i,\pi_i(k)}$ that are less than ϵ to ϵ . We can verify that the quantization function in (5) is equivalent to that in (2).

To address the issue that the gradient of the $\chi_{0.5}^{(k)}(x)$ is almost zero everywhere, we adopt the straight-through estimator (STE) approximation for the backward pass of the step function $\chi_{0.5}^{(k)}(x)$. This is defined as follows:

$$\frac{\partial \chi_{0.5}^{(k)}(x)}{\partial x} = \begin{cases} 1 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise.} \end{cases} \quad (7)$$

This means that the gradient for parameters within the $[0, 1]$ interval is 1, while the gradient for parameters outside this interval is 0. The gradients for the rest of the operations in the quantization function $Q(\cdot)$ remain unchanged. By applying the chain rule, we can obtain $\frac{\partial \mathbf{W}^Q}{\partial \mathbf{S}}, \frac{\partial \mathbf{W}^Q}{\partial \mathbf{V}}, \frac{\partial \mathbf{W}^Q}{\partial \mathbf{B}}$ to optimize the quantization parameters $\mathbf{S}, \mathbf{V}, \mathbf{B}$ with gradient descent.

3.3.2 Reparameterization of Quantization Parameters

During the gradient descent process, we find it challenging to directly learn the scaling vectors $\mathcal{S}$ for all layers. The main reason is that the values can vary significantly among different layers, and even within the same layer (e.g., in the fc1 layer), as shown in Figure 2. Directly learning $\mathcal{S}$ can result in excessive gradient updates for some weights while being too small for others, leading to difficulty in convergence. As in OmniQuant (Shao et al., 2024),

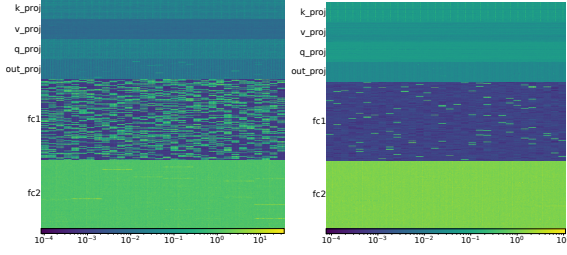


Figure 2: Visualization of the scaling vector $\mathcal{S}$ with AWQ, shown in the log scale. 3-bit quantization of a Transformer block in OPT-1.3B model. Left image: block 1; Right image: block 10.

we propose a reparameterization strategy to address the divergence issue caused by the large differences in $\mathcal{S}$ values. The reparameterization of $\mathcal{S}_{*,i}$ for the i th group of weights can be written as follows:

$$\begin{aligned} \mathcal{S}_{*,i} &= g_S(\bar{\mathcal{S}}_{*,i}, \mathbf{W}_i) \\ &= \tanh(\bar{\mathcal{S}}_{*,i}) \left(\frac{\max(\mathbf{W}_i) - \min(\mathbf{W}_i)}{2} \right). \end{aligned} \quad (8)$$

Here, different groups of weights have different reparameterization coefficients $\frac{\max(\mathbf{W}_i) - \min(\mathbf{W}_i)}{2}$, decoupling the weight range from $\mathcal{S}$. After reparameterization, the parameter to be optimized changes from $\mathcal{S}$ to $\bar{\mathcal{S}}$, which is relatively uniform across different layers and can improve the consistency among gradient magnitudes to facilitate convergence. The use of $\tanh(\cdot)$ constrains the range of $\bar{\mathcal{S}}$, and subsequently prevents $\mathcal{S}$ from exceeding the maximum range of a group of weights.

For the reparameterization of the quantization offsets $\mathbf{B}$, we employ a similar method as that for the reparameterization of $\mathcal{S}$. But we need to solve an extra issue. More specifically, according to previous studies (Lin et al., 2023; Frantar et al., 2023), it is generally necessary to include the value 0 in the set of quantized values. The main reason is that many weights in the model are close to 0, and hence excluding 0 from the quantized values would lead to significant quantization loss and severely degrade the accuracy of the final model. Assume the current offsets are $\mathbf{B}'$. For the offsets $\mathbf{B}'_i$ of the i th group, the reparameterized form is as follows:

$$\mathbf{B}'_i = g_S(\bar{\mathbf{B}}_i, \mathbf{W}_i). \quad (9)$$

A new substitute offset $\mathbf{B}_i$ must be recalculated in each optimization iteration based on $\mathbf{B}'_i$, ensuring that the value 0 is included in the codebook $\mathbf{C}_i$ when using $\mathbf{B}_i$. The offset calculation formula is

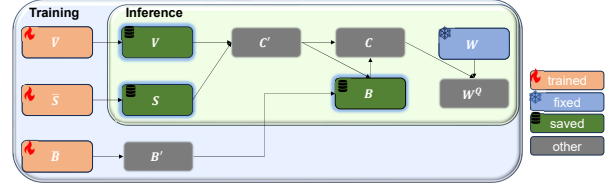


Figure 3: Training and inference process of LCQ with reparameterization.

as follows: $\mathbf{B}_i = Q(\mathbf{B}'_i, \mathbf{C}'_i)$, where $\mathbf{C}'_i = \mathbf{S}_i^T \mathbf{V}$. This ensures that $\mathbf{B}_i$ is definitely an element of $\mathbf{C}'_i$, and consequently, 0 is assuredly an element of the quantization codebook $\mathbf{C}_i = \mathbf{S}_i^T \mathbf{V} - \mathbf{B}_i$.

We also reparameterize $\mathbf{V}$. As the range of weights has already been represented by $\mathcal{S}$, $\mathbf{V}$ does not need to represent the range of weights. We only need to use the $\tanh(\cdot)$ function to constrain the range of $\mathbf{V}$ to be in $[-1, 1]$:

$$\mathbf{V} = g_V(\bar{\mathbf{V}}) = \tanh(\bar{\mathbf{V}}). \quad (10)$$

Although the reparameterization strategy might result in a codebook in which the maximum range could exceed the maximum range of the group weights, we can constrain the initial maximum range of the codebook to be within the maximum range of the group weights during initialization. Subsequent optimization during gradient descent will then fine-tune the codebook within the vicinity of the initial values.

The training and inference process of LCQ with reparameterization is shown in Figure 3.

3.3.3 Initialization of Quantization Parameters

We use the results of AWQ (Lin et al., 2023) to initialize $\bar{\mathcal{S}}_1, \bar{\mathbf{B}}, \bar{\mathbf{V}}_1$. From the results of AWQ, we can obtain the parameters $\mathcal{S}_1, \mathbf{B}', \mathbf{V}_1$. To get the corresponding reparameterization results, we calculate $\bar{\mathcal{S}}_1, \bar{\mathbf{B}}, \bar{\mathbf{V}}_1$ based on the inverse functions corresponding to (8), (9), and (10). Specifically, AWQ (Lin et al., 2023) optimizes the equivalent transformation parameters and the clipping parameters of each linear layer. For the equivalent transformation parameters, we apply the optimized equivalent transformation parameters of AWQ to scale the weights of each linear layer. For the clipping parameters, AWQ searches for the clipping parameters for each group of weights and then clip the weights. The range and mid-point of the clipped weights correspond to $\mathcal{S}_1, \mathbf{B}'$. In

addition, $\mathbf{V}_1$ is a vector with uniform interval, i.e., $\mathbf{V}_1 = [-1, -1 + \frac{2}{2^b-1}, \dots, 1 - \frac{2}{2^b-1}, 1]$ in AWQ. For the remaining parameters, $\{\mathbf{S}_2, \mathbf{S}_3, \dots, \mathbf{S}_{N_D}\}$ is initialized to zero, and $\{\mathbf{V}_2, \mathbf{V}_3, \dots, \mathbf{V}_{N_D}\}$ is initialized with equidistant points from a standard Gaussian distribution. This ensures that the initialization of LCQ corresponds exactly to the results of AWQ, making the following optimization process easier to converge to a good result.

3.4 Double Quantization

When $N_D > 1$, LCQ will have more parameters than rank-one codebook based methods like AWQ, which will result in extra storage cost. To mitigate the extra storage cost associated with the enlarged $\mathbf{S}$ and $\mathbf{V}$, we adopt a double quantization strategy as in QLoRA (Dettmers et al., 2023) to further quantize $\mathbf{S}$ and $\mathbf{V}$. Taking $\mathbf{S}$ as an example, with double quantization, the optimization goal is to minimize the quantization error of $\mathbf{S}$:

$$\min_{\mathbf{S}^{dq}, \mathbf{V}^{dq}, \mathbf{B}^{dq}} \|\mathcal{Q}_{\Phi(\mathbf{S}^{dq}, \mathbf{V}^{dq}, \mathbf{B}^{dq})}(\mathbf{S}) - \mathbf{S}\|_2^2 \quad (11)$$

Here, $\mathbf{S}^{dq}, \mathbf{V}^{dq}, \mathbf{B}^{dq}$ represent the parameters for double quantization. We adopt uniform quantization for double quantization. Like AWQ, we employ grid search to find the optimal $\mathbf{S}^{dq}, \mathbf{V}^{dq}, \mathbf{B}^{dq}$. More specifically, grid search is first used to identify the double quantization parameters for $\mathbf{S}$, which are then fixed. Subsequently, grid search is applied to determine the double quantization parameters for $\mathbf{V}$. As this process is independent of the input, the overall computational (time) cost is negligible.

The whole optimization procedure for LCQ is outlined in Algorithm 1. We sequentially quantize each Transformer block weights $\mathcal{W}_l$.

4 Experiments

4.1 Experimental Setup

Calibration Dataset: As in AWQ (Lin et al., 2023), 128 samples randomly selected from Pile dataset (Merity et al., 2016) are used as the calibration dataset. The sequence length of each sample is 2048.

Quantization Hyperparameters: We adopt group quantization with two settings for group size: "G128" denotes a group size of 128, and "G-1" denotes that the group size is equal to the size of the channel. N_D is set to 2 by default. More training details are in Appendix A.

Algorithm 1 Optimization Procedure for LCQ

Input: Pretrained LLM, calibration dataset $\mathcal{X}$.

Output: Quantization parameters for each Transformer block: $\mathcal{S}, \mathcal{V}, \mathcal{B}$.

Initialize full-precision input features: $\mathcal{X}_{fp} = \mathcal{X}$;

for $l = 1$ **to** N_{block} **do**

Initialize $\bar{\mathcal{S}}, \bar{\mathcal{V}}, \bar{\mathcal{B}}$ according to Section 3.3.3;

for $t = 1$ **to** T **do**

Sample a mini-batch from $\mathcal{X}_{fp}$;

According to Section 3.3.2, $\mathcal{S}, \mathcal{B}', \mathcal{V}$ are computed from $\bar{\mathcal{S}}, \bar{\mathcal{B}}, \bar{\mathcal{V}}$. $\mathcal{B}$ is obtained from $\mathcal{B}'$;

Compute loss according to (4), and update parameters $\bar{\mathcal{V}}, \bar{\mathcal{S}}, \bar{\mathcal{B}}$ using gradient back-propagation with gradient approximation in (7);

end for

Perform double quantization for $\mathcal{S}, \mathcal{V}$ according to (11), use grid search to find the optimal double quantization codebook, and save the parameters $\mathcal{S}, \mathcal{V}, \mathcal{B}$ of the current block;

Compute the full-precision input features for the next block: $\mathcal{X}_{fp} = h(\mathcal{X}_{fp}, \mathcal{W}_l)$;

end for

4.2 Comparison with Baselines

We choose AWQ (Lin et al., 2023) and OmniQuant (Shao et al., 2024) as baselines for comparison, because they are the most related works and have achieved state-of-the-art performance.

We first compare LCQ with baselines on the task of natural language generation, using OPT (Zhang et al., 2022) and LLaMA (Touvron et al., 2023) as pre-trained models. As in AWQ (Lin et al., 2023), we use three datasets wikitext2 (Merity et al., 2017), c4 (Raffel et al., 2020) and ptb (Marcus et al., 1994), with the perplexity (ppl) metric for evaluating the ability of the model to predict the next word (i.e., generative capability). The *retention rate*, defined as the rate between the remaining parameters to the original parameters, reflects the degree of model compression. The higher the retention rate is, the lower the degree of model compression will be. Table 1, Table 2 and Table 3 show the results on the three datasets, respectively. "fp16" denotes the original non-compressed model with 16-bit full-precision representation. "W2 G128" denotes a 2-bit weight quantization with group size of 128. Other notations are defined similarly. We

Table 1: Natural language generation ability on wikitext2, with perplexity as the metric. The best is shown in bold.

Config	Method	Retention Rate	OPT-125M	OPT-1.3B	LLaMA-7B
fp16	-	1	27.65	14.62	5.68
W2 G128	AWQ (Lin et al., 2023)	0.134	340.56	42.72	12.90
	OmniQuant (Shao et al., 2024)	0.134	117.25	34.69	12.33
	LCQ	0.138	68.75	25.28	9.14
W3 G-1	AWQ (Lin et al., 2023)	0.188	57.46	24.67	8.43
	OmniQuant (Shao et al., 2024)	0.188	41.50	18.85	6.73
	LCQ	0.188	36.88	17.54	6.41
W3 G128	AWQ (Lin et al., 2023)	0.197	35.99	16.41	6.31
	OmniQuant (Shao et al., 2024)	0.197	35.23	16.37	6.30
	LCQ	0.202	32.47	15.97	6.17

Table 2: Natural language generation ability on c4, with perplexity as the metric. The best is shown in bold.

Config	Method	Retention Rate	OPT-125M	OPT-1.3B	LLaMA-7B
fp16	-	1	26.56	16.07	7.34
W2 G128	AWQ (Lin et al., 2023)	0.134	246.47	40.63	16.55
	OmniQuant (Shao et al., 2024)	0.134	82.47	31.59	14.79
	LCQ	0.138	54.20	25.31	11.44
W3 G-1	AWQ (Lin et al., 2023)	0.188	48.83	24.55	11.13
	OmniQuant (Shao et al., 2024)	0.188	36.36	19.82	8.73
	LCQ	0.188	33.81	18.76	8.35
W3 G128	AWQ (Lin et al., 2023)	0.197	32.63	17.80	8.17
	OmniQuant (Shao et al., 2024)	0.197	31.92	17.69	8.18
	LCQ	0.202	30.12	17.30	8.02

can find that LCQ achieves better accuracy than the baselines. In particular, in the case of 2-bit quantization, LCQ achieves significantly better accuracy than baselines. Furthermore, LCQ has a comparable retention rate with baselines, which means that the increase of N_D brings a negligibly extra storage cost.

We also compare LCQ with baselines on zero-shot tasks using three widely used datasets PIQA (Bisk et al., 2020), WinoGrande (Sakaguchi et al., 2020) and ARC_easy (Clark et al., 2018). Table 4 shows the accuracy of the quantized model for LLaMA-7B. We can find that LCQ achieves better accuracy than baselines on zero-shot tasks for all datasets. In particular, in the case of 2-bit quantization, the improvement is significant compared to baselines. The performance on zero-shot tasks also verifies that our LCQ method does not over-fit the calibration dataset and can maintain the generalization capability of the original model on zero-shot tasks.

4.3 Hyperparameter Sensitivity

First, we investigate the impact of different initialization methods for LCQ. We only perform experiment about the initialization for V_2 , and fix the initialization of other quantization parameters as described in Section 3.3.3. We compare three initialization methods: rand, uniform and normal. *rand* denotes the method that randomly samples points

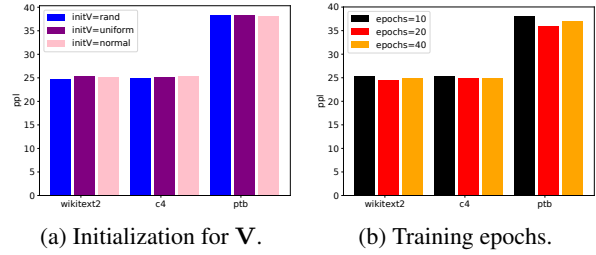


Figure 4: Hyperparameter sensitivity with “W2 G128” for OPT-1.3B.

from the Gaussian distribution $\mathcal{N}(0, 0.1)$ and then sorts them from smallest to largest. *uniform* denotes the method using equidistant quantiles of a uniform distribution. *normal* denotes the method using equidistant quantiles of a Gaussian distribution $\mathcal{N}(0, 0.1)$ (Dettmers et al., 2023). Figure 4a shows the result. We can find that different initialization methods do not have a significant impact on the performance. Hence, LCQ is robust to the initialization methods for V_2 . By default, we use *normal* for initialization to avoid extra hyperparameter settings.

We also study the impact of the number of training epochs. Figure 4b shows the result. We can find that 10 training epochs can achieve results comparable to those of 40 epochs. Hence, to expedite the training process, we adopt 10 training epochs by default.

The result about N_V is in Figure 5a. We set N_V

Table 3: Natural language generation ability on ptb, with perplexity as the metric. The best is shown in bold.

Config	Method	Retention Rate	OPT-125M	OPT-1.3B	LLaMA-7B
fp16	-	1	38.99	20.29	41.15
W2 G128	AWQ (Lin et al., 2023)	0.134	516.64	73.33	198.35
	OmniQuant (Shao et al., 2024)	0.134	163.66	49.46	214.53
	LCQ	0.138	89.73	38.12	103.84
W3 G-1	AWQ (Lin et al., 2023)	0.188	86.42	34.65	76.75
	OmniQuant (Shao et al., 2024)	0.188	58.22	26.77	56.33
	LCQ	0.188	53.72	25.38	53.86
W3 G128	AWQ (Lin et al., 2023)	0.197	50.38	23.23	48.01
	OmniQuant (Shao et al., 2024)	0.197	48.78	22.85	52.62
	LCQ	0.202	46.75	22.40	49.84

Table 4: Zero-shot performance of the quantized model for LLaMA-7B, with accuracy as the metric. The best is shown in bold.

Config	Method	PIQA	WinoGrande	ARC_easy
fp16	-	78.40	66.93	67.34
W2 G128	AWQ (Lin et al., 2023)	69.21	55.80	52.74
	OmniQuant (Shao et al., 2024)	66.92	53.20	49.37
	LCQ	72.36	60.30	59.13
W3 G128	AWQ (Lin et al., 2023)	76.33	65.19	65.07
	OmniQuant (Shao et al., 2024)	76.33	64.96	64.44
	LCQ	77.31	66.38	67.13

according to different weight size N_W . The larger the N_V , the finer the quantization granularity. In particular, $N_V = 1$ corresponds to the coarsest granularity, indicating that the whole linear layer corresponds a single $\mathbf{V}$. We can find that reducing N_V can bring a slight improvement in model accuracy but also introduces some additional storage cost. To balance model accuracy and storage cost, we set $N_V = \frac{N_W}{G \times 32}$ by default.

The number of bits for double quantization will also affect the performance. Figure 5b shows the result. In particular, $\mathbf{S}_1$ is not quantized and kept in fp16, consistent with baseline methods. We can find that when using 4-bit quantization for $\mathbf{S}$ and 8-bit quantization for $\mathbf{V}$, the accuracy loss is minor or even negligible compared to methods without double quantization. However, when the number of quantization bits for $\mathbf{V}$ is reduced to 4, the accuracy loss is more significant. Hence, we use 4-bit quantization for $\mathbf{S}$ and 8-bit quantization for $\mathbf{V}$ by default. Double quantization is performed in groups of 16 elements by default.

We also explore the impact of N_D on model performance, the result of which is shown in Table 5. We can find that compared to the results with $N_D = 1$, the results with $N_D = 2$ improve significantly, which verifies that increasing the rank of the codebook can lead to better accuracy. Further increasing N_D can further improve the accuracy, but will also incur more storage cost. Hence, we set N_D to 2 by default. Furthermore, we can also find

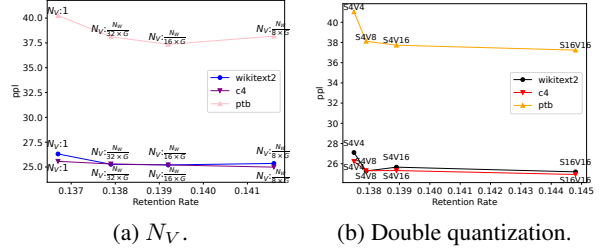


Figure 5: Hyperparameter sensitivity with ‘‘W2 G128’’ for OPT-1.3B.

Table 5: Perplexity on wikitext2 when using different N_D and whether fixing $\mathbf{S}_1, \mathbf{B}, \mathbf{V}_1$, with ‘‘W2 G128’’ for OPT-1.3B.

N_D	Retention Rate	Fix	OPT-125M	OPT-1.3B
1	0.135	✓	340.56	42.72
1	0.135	×	102.39	30.10
2	0.137	✓	76.98	27.09
2	0.138	×	68.75	25.28
3	0.141	✓	65.52	25.15
3	0.141	×	61.49	24.82

that although the optimized parameters of AWQ are used to initialize $\mathbf{S}_1, \mathbf{V}_1, \mathbf{B}$ in LCQ, further learning (optimizing) these parameters can achieve better accuracy than fixing them.

5 Conclusion

In this paper, we propose a novel weight quantization method called LCQ for compressing LLMs. Experiments show that LCQ can outperform existing methods to achieve the best performance.

Limitations

We have only conducted experiments on uni-modal models and have not yet validated our method on multi-modal models. We will validate our method on multi-modal models in the future.

Ethics Statement

Our paper will not cause ethical issues and the datasets used in this paper are all public datasets.

References

- Haoli Bai, Lu Hou, Lifeng Shang, Xin Jiang, Irwin King, and Michael R. Lyu. 2022. [Towards efficient post-training quantization of pre-trained language models](#). In *NeurIPS*, pages 1405–1418.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. [PIQA: reasoning about physical commonsense in natural language](#). In *AAAI*, pages 7432–7439.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. [Language models are few-shot learners](#). In *NeurIPS*, pages 1877–1901.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. [Think you have solved question answering? try arc, the A12 reasoning challenge](#). *CoRR*, abs/1803.05457.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. [Llm.int8\(\): 8-bit matrix multiplication for transformers at scale](#). In *NeurIPS*, pages 30318–30332.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. [Qlora: Efficient finetuning of quantized llms](#). In *NeurIPS*.
- Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. 2024. [Spqr: A sparse-quantized representation for near-lossless LLM weight compression](#). In *ICLR*.
- Elias Frantar and Dan Alistarh. 2022. [Optimal brain compression: A framework for accurate post-training quantization and pruning](#). In *NeurIPS*, pages 4475–4488.

- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. [GPTQ: accurate post-training quantization for generative pre-trained transformers](#). In *ICLR*.
- Song Han, Huizi Mao, and William J. Dally. 2016. [Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding](#). In *ICLR*.
- Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joon-suk Park, Kang Min Yoo, Se Jung Kwon, and Dong-soo Lee. 2023. [Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization](#). In *NeurIPS*.
- Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. 2024. [OWQ: lessons learned from activation outliers for weight quantization in large language models](#). In *AAAI*.
- Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. 2024. [Loftq: Lora-fine-tuning-aware quantization for large language models](#). In *ICLR*.
- Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. [BRECQ: pushing the limit of post-training quantization by block reconstruction](#). In *ICLR*.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. [AWQ: activation-aware weight quantization for LLM compression and acceleration](#). *CoRR*, abs/2306.00978.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. [Visual instruction tuning](#). *CoRR*, abs/2304.08485.
- Zechun Liu, Kwang-Ting Cheng, Dong Huang, Eric P. Xing, and Zhiqiang Shen. 2022. [Nonuniform-to-uniform quantization: Towards accurate quantization via generalized straight-through estimation](#). In *CVPR*, pages 4932–4942.
- Ilya Loshchilov and Frank Hutter. 2017. [SGDR: stochastic gradient descent with warm restarts](#). In *ICLR*.
- Ilya Loshchilov and Frank Hutter. 2019. [Decoupled weight decay regularization](#). In *ICLR*.
- Mitchell P. Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. 1994. [The penn treebank: Annotating predicate argument structure](#). In *Human Language Technology*, pages 114–119.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. [Pointer sentinel mixture models](#). *CoRR*, abs/1609.07843.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *ICLR*.

680	Sangyun Oh, Hyeonuk Sim, Jounghyun Kim, and	Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang,	737
681	Jongeun Lee. 2022. Non-uniform step size quan-	Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022.	738
682	tization for accurate post-training quantization. In	Zeroquant: Efficient and affordable post-training	739
683	ECCV , pages 658–673.	quantization for large-scale transformers. In	740
684	Adam Paszke, Sam Gross, Francisco Massa, Adam	NeurIPS , pages 27168–27183.	741
685	Lerer, James Bradbury, Gregory Chanan, Trevor	Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xing-	742
686	Killeen, Zeming Lin, Natalia Gimelshein, Luca	gang Wang, Yuzhang Shang, Guangyu Sun, Qiang	743
687	Antiga, Alban Desmaison, Andreas Köpf, Edward Z.	Wu, Jiayang Wu, and Bingzhe Wu. 2023. RPTQ:	744
688	Yang, Zachary DeVito, Martin Raison, Alykhan Te-	reorder-based post-training quantization for large lan-	745
689	jani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang,	guage models. <i>CoRR</i> , abs/2304.01089.	746
690	Junjie Bai, and Soumith Chintala. 2019. Pytorch:	Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang,	747
691	An imperative style, high-performance deep learning	Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu,	748
692	library. In <i>NeurIPS</i> , pages 8024–8035.	Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma,	749
693	Colin Raffel, Noam Shazeer, Adam Roberts, Katherine	Yufei Xue, Jidong Zhai, Wenguang Chen, Zhiyuan	750
694	Lee, Sharan Narang, Michael Matena, Yanqi Zhou,	Liu, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023.	751
695	Wei Li, and Peter J. Liu. 2020. Exploring the limits	GLM-130B: an open bilingual pre-trained model. In	752
696	of transfer learning with a unified text-to-text trans-	<i>ICLR</i> .	753
697	former. <i>JMLR</i> , 21:140:1–140:67.	Susan Zhang, Stephen Roller, Naman Goyal, Mikel	754
698	Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavat-	Artetxe, Moya Chen, Shuohui Chen, Christopher	755
699	ula, and Yejin Choi. 2020. Winogrande: An adver-	Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin,	756
700	sarial winograd schema challenge at scale. In <i>AAAI</i> ,	Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shus-	757
701	pages 8732–8740.	ter, Daniel Simig, Punit Singh Koura, Anjali Srid-	758
702	Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng	har, Tianlu Wang, and Luke Zettlemoyer. 2022.	759
703	Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng	OPT: open pre-trained transformer language mod-	760
704	Gao, Yu Qiao, and Ping Luo. 2024. Omniquant:	els. <i>CoRR</i> , abs/2205.01068.	761
705	Omnidirectionally calibrated quantization for large		
706	language models. In <i>ICLR</i> .		
707	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier		
708	Martinet, Marie-Anne Lachaux, Timothée Lacroix,		
709	Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal		
710	Azhar, Aurélien Rodriguez, Armand Joulin, Edouard		
711	Grave, and Guillaume Lample. 2023. Llama: Open		
712	and efficient foundation language models. <i>CoRR</i> ,		
713	abs/2302.13971.		
714	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob		
715	Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz		
716	Kaiser, and Illia Polosukhin. 2017. Attention is all		
717	you need. In <i>NeurIPS</i> , pages 5998–6008.		
718	Xiuying Wei, Ruihao Gong, Yuhang Li, Xianglong Liu,		
719	and Fengwei Yu. 2022. Qdrop: Randomly dropping		
720	quantization for extremely low-bit post-training quan-		
721	tization. In <i>ICLR</i> .		
722	Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo		
723	Zhang, Ruihao Gong, Jinyang Guo, and Xianglong		
724	Liu. 2023. Outlier suppression+: Accurate quanti-		
725	zation of large language models by equivalent and		
726	optimal shifting and scaling. <i>CoRR</i> , abs/2304.09145.		
727	Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu,		
728	Julien Demouth, and Song Han. 2023. Smoothquant:		
729	Accurate and efficient post-training quantization for		
730	large language models. In <i>ICML</i> , pages 38087–		
731	38099.		
732	Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng		
733	Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng		
734	Zhang, and Qi Tian. 2024. Qa-lora: Quantization-		
735	aware low-rank adaptation of large language models.		
736	In <i>ICLR</i> .		

A Training Details

We perform experiments with PyTorch (Paszke et al., 2019) on an RTX A6000 GPU card of 48GB memory, and use mixed-precision optimization to reduce the memory and computational cost during training. The ADAMW optimizer (Loshchilov and Hutter, 2019) is used, with an initial learning rate set to 0.01, and the weight decay factor set to 0. The learning rate is gradually reduced using a cosine annealing schedule (Loshchilov and Hutter, 2017) over a total of 10 training epochs. The batch size for training is set to 4, and gradient accumulation is employed to prevent memory overflow. The linear layer weights of all Transformer blocks are quantized. The embedding layer and the output layer are not quantized.